



Developing Open LLM applications with



Apache OpenServerless

Lesson 6

Vector Database

Vector Database

- Vector Database
- Embedding and Search
- Importing PDF



Vector Database

Milvus: concepts

- Milvus is a No-SQL database optimized for **vector searches**
- Multiple **databases**, each database has multiple **collections**
- Each collection has a **schema** and **indexes**
- Vector Search:
 - finding **similarities** in a dataset looking at their **numeric representation**
 - text is transformed in a numeric representatino using an **embedding model** then stored

Access Milvus

```
import os
from pymilvus import MilvusClient

uri = f"http://{os.getenv('MILVUS_HOST')}:"
token = os.getenv("MILVUS_TOKEN")
db_name = os.getenv("MILVUS_DB_NAME")
client = MilvusClient(uri=uri, token=token, db_name=db_name)
```

- Using Milvus

```
client.list_collections()
client.drop_collection("test")
```

Collections

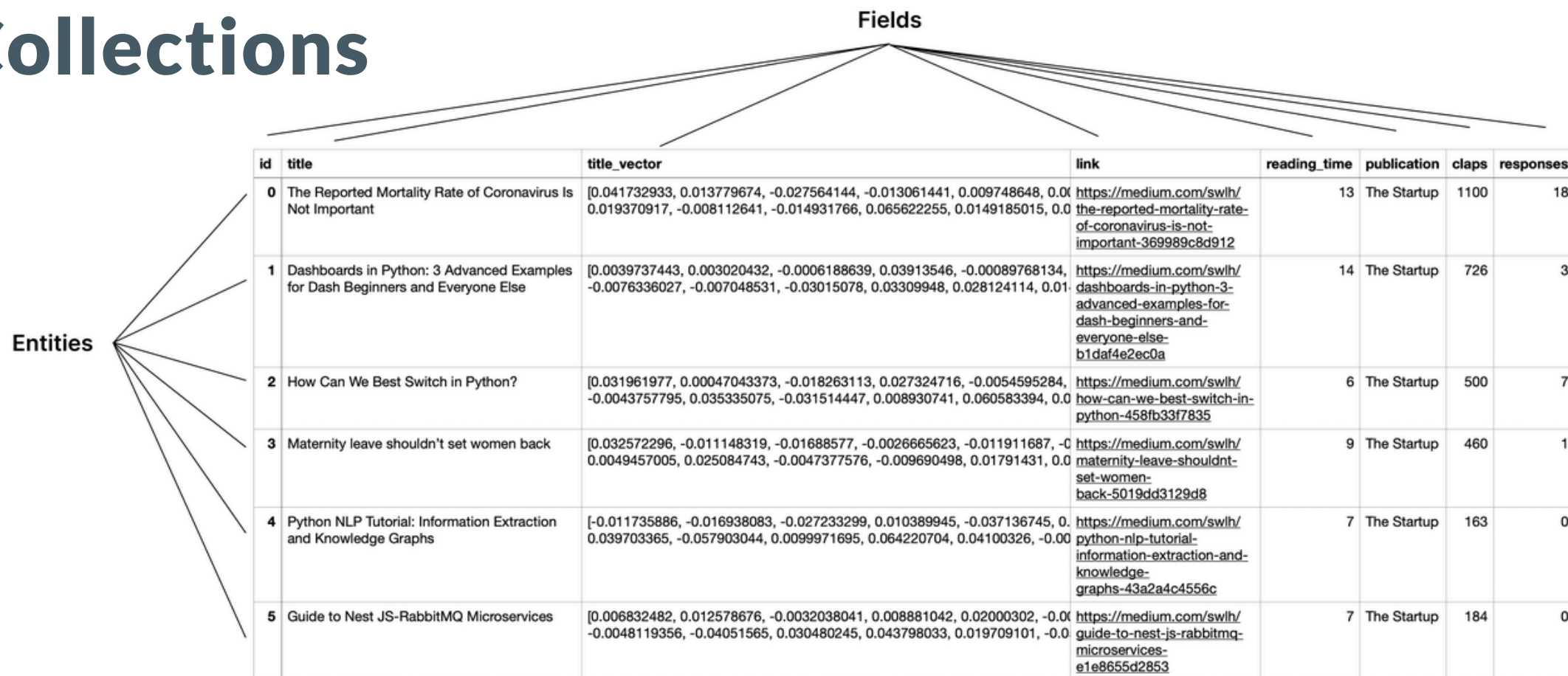


Figure 1: A Medium article collection in Zilliz Cloud database.

Create Schema

- Parameters

```
from pymilvus import DataType
COLLECTION = "test"
DIMENSION=1024
```

- Define Schema

```
schema = client.create_schema()
schema.add_field(field_name="id", datatype=DataType.INT64, is_primary=True, auto_id=True)
schema.add_field(field_name="text", datatype=DataType.VARCHAR, max_length=DIMENSION)
schema.add_field(field_name="embeddings", datatype=DataType.FLOAT_VECTOR, dim=DIMENSION)
```

Create Index and Collection

- Define Index

```
index_params = client.prepare_index_params()  
index_params.add_index("embeddings", index_type="AUTOINDEX", metric_type="IP")
```

- Create Collection with Index and Schema

```
client.create_collection(  
    collection_name=COLLECTION,  
    schema=schema, index_params=index_params)
```


Insert

```
text = "Hello World"
vec = [float(i) for i in range(0,DIMENSION)] # TO BE REPLACED with embedding
client.insert(COLLECTION, {"text":text, "embeddings": vec})
```

Retrieve

```
qit = client.query_iterator(collection_name=COLLECTION, batchSize=2, output_fields=["text"])
res = qit.next()
print(res[0].get("text"))
```

Hello World

Embedding

Embedding

- Use an embedding model

```
import sys, requests as req
MODEL="mxbai-embed-large:latest"
DIMENSION=1024
```

- Invoke the embedding API

```
inp = "Hello World"
url = f"https://{os.getenv("AUTH")}@{os.getenv("OLLAMA_HOST")}/api/embeddings"
msg = { "model": MODEL, "prompt": inp, "stream": False }
res = req.post(url, json=msg).json()
out = res.get('embedding', [])
```

VectorDB with embedding

- Using the `VectorDB` class

```
sys.path.append("packages/vdb/load")  
import vdb  
db = vdb.VectorDB({})
```

- Insert text with embedding

```
db.insert("Hello World")  
db.insert("This is a test")  
db.insert("This is another test")  
db.insert("Testing")
```

Vector Search

- Prepare

```
text = "Test"  
vec = db.embed(text)
```

- Execute the actual search

```
cur = client.search(collection_name=COLLECTION, # collection  
                     search_params={"metric_type": "IP"}, # how to measure distance  
                     anns_field="embeddings", # where to search  
                     data=[vec], # what to search  
                     output_fields=["text"] # field to return  
)
```

Vector Search results

```
for item in cur[0]:  
    dist = item.get('distance', 0)  
    text = item.get("entity", {}).get("text", "")  
    print(dist, text)
```

271.28466796875 Testing

252.88241577148438 This is a test

231.84872436523438 This is another test

181.67494201660156 Hello World

Note that output it is ordered by **distance**

Pinocchio - Nuvolaris

msciab.openserverless.dev

Pinocchio

Puzzle

Hello

Stream

Redis

Milvus

S3

Ollama

RAG

Load

Test

Hello

Demo

Vision

Form

Store

msciab Load Side View

BOT 21:26

Inserted 456142061011228407

YOU 21:27

This is another test

BOT 21:27

Inserted 456142061011228409

YOU 21:27

*test

BOT 21:27

Found:
(271.28) Testing
(252.88) This is a test
(231.85) This is another test
(203.04) Hello

Enter your message... Send

vdb/load action

```
!code packages/vdb/load/vdb.py  
!code packages/vdb/load/load.py
```

PDF Import

Extract content from PDF files

Pages from a PDF

```
import pymupdf
doc = pymupdf.open("lessons/bitcoin.pdf") # load a file
text = page[0].get_text()                # extract text from page
```

Split in sentences

```
import nltk.data
from nltk.tokenize import sent_tokenize
nltk.data.find('tokenizers/punkt') # load tokenized model
sentences = sent_tokenize(text)    # extract sentences from text
enum = enumerate(sentences, 1)     # enumerate
sent = next(enum)                  # extract one sentence
```

ops ai loader

```
!ops ai loader
```

```
ai loader [--action=<action>] <file>...
```

```
!ops ai loader lessons/bitcoin.pdf
```

```
>>> converting lessons/bitcoin.pdf  
saved lessons/bitcoin.pdf.txt
```